ASP.NET Core Filters

Architecture, Implementation, and Modern Development Context

1. What are Filters?

Filters in ASP.NET Core allow developers to run code immediately before or after specific stages of the request processing pipeline. Unlike middleware, which operates on every request at the HTTP protocol level, filters execute **within the endpoint routing and MVC invocation pipeline**.

Because they run inside the MVC framework, filters have deep access to MVC constructs, such as:

- The underlying **Controller** instance.
- The **Action Arguments** (the parameters being passed to the method).
- The **ModelBinding** state and validation results.
- The **IActionResult** before it is formatted and written to the response stream.

2. Middleware vs. Filters: Architectural Distinction

Understanding when to use Middleware versus when to use Filters is a critical architectural decision. Misplacing logic leads to either lack of context (Middleware trying to read MVC data) or bypassed execution (Filters not running because routing failed).

```
Client Request
  ↓
[ Middleware 1 (e.g., Error Handling) ]
  ↓
[ Middleware 2 (e.g., Routing) ]
  ↓
[ Endpoint Middleware (Executes matched route) ]
      │
      ├─> [ Authorization Filters ]
      │       ↓
      ├─> [ Resource Filters ]
      │       ↓
      │       (Model Binding Occurs Here)
      │       ↓
      ├─> [ Action Filters ]
      │       ↓
      │   [ Controller Action Method ]
      │       ↓
```

```
      ├─> [ Result Filters ]
      │       ↓
      │    (Result Execution / JSON Serialization)
      │
   ↓ (Returns to Middleware Pipeline)
[ Middleware 1 Post-Processing ]
   ↓
Client Response
```

Use Middleware When...	Use Filters When...
- Logic applies to **all** requests (e.g., CSS files, 404s, APIs). - You need to modify the raw HTTP Request/ Response bodies. - You are handling global performance metrics, CORS, or raw URL rewriting.	- Logic is specific to Controllers or Actions. - You need access to the method parameters or `ModelState`. - You need to format or inspect the `IActionResult` before it is sent. - You want to apply logic selectively using Attributes (e.g., `[ValidateModel]`).

3. The Five Types of Filters

1. **Authorization Filters:** Run first. Used to determine if the user is authorized for the request. *Note: In modern ASP.NET Core, the built-in Authorization Middleware and `[Authorize]` attribute are preferred over writing custom Authorization filters.*
2. **Resource Filters:** Run after authorization, but before Model Binding. Useful for caching or short-circuiting the pipeline early if a cached response exists.
3. **Action Filters:** Run immediately before and after the controller action method is called. They can inspect and manipulate the arguments passed into an action, as well as the result returned from it.
4. **Exception Filters:** Apply global policies to unhandled exceptions occurring *within the controller creation, model binding, action filters, or action method*.
5. **Result Filters:** Run immediately before and after the execution of the action result (e.g., right before the JSON is serialized and sent to the client).

4. Are Filters Used in Modern Development (.NET 7/8+)?

The role of filters has evolved significantly in modern ASP.NET Core development, particularly with the introduction of Minimal APIs and new middleware capabilities. It is critical to understand where they are still the correct architectural choice.

Controller-Based APIs (Still Heavily Used)

In traditional Controller-based APIs, **Action Filters** and **Result Filters** remain standard practice. They are the preferred mechanism for cross-cutting concerns that require context, such as:

- Validating incoming DTOs against business rules that require database access (running before the action).
- Standardizing API response formats (wrapping a returned object into a standard Envelope response).

Minimal APIs (IEndpointFilter)

Historically, Minimal APIs did not support MVC filters because they bypass the MVC framework entirely. However, starting in .NET 7, Microsoft introduced **Endpoint Filters** (`IEndpointFilter`). This brings filter-like capabilities to Minimal APIs, allowing developers to intercept the request, read parameters, and modify the result.

```
app.MapPost("/api/vehicles", (VehicleDto vehicle) => { ... })
   .AddEndpointFilter(async (invocationContext, next) =>
   {
       var vehicle = invocationContext.GetArgument<VehicleDto>(0);
       // Perform validation on the argument
       return await next(invocationContext);
   });
```

The Shift in Exception Handling (Trade-off Analysis)

Exception Filters are generally **discouraged** in modern API design.

Why? An Exception Filter only catches exceptions that occur *inside* the MVC pipeline. If an exception occurs during routing, inside custom middleware, or during response serialization, the Exception Filter will not catch it, leading to inconsistent error responses (e.g., returning HTML instead of a JSON ProblemDetails object).

Preferred Approach: Modern APIs use Global Exception Middleware or the .NET 8 `IExceptionHandler` interface, which sits at the very top of the pipeline and catches *everything*, ensuring a 100% consistent API error contract.

5. Implementation Example: Action Filter

This is a technical implementation of a modern asynchronous Action Filter used to validate arguments before the Controller action executes.

```
public class ValidateVehicleExistsFilter : IAsyncActionFilter
{
    private readonly IVehicleRepository _repository;

    // Services can be injected via constructor
    public ValidateVehicleExistsFilter(IVehicleRepository repository)
    {
        _repository = repository;
    }

    public async Task OnActionExecutionAsync(ActionExecutingContext context,
ActionExecutionDelegate next)
    {
        // 1. Pre-processing: Executes before the controller action
        if (context.ActionArguments.TryGetValue("id", out var idObj) && idObj is int id)
        {
            var exists = await _repository.VehicleExistsAsync(id);
            if (!exists)
            {
                // Short-circuit the pipeline: The controller method is never called
                context.Result = new NotFoundObjectResult(new { Message = $"Vehicle {id}
not found." });
                return;
            }
        }

        // 2. Execute the action (and subsequent filters)
        var resultContext = await next();

        // 3. Post-processing: Executes after the controller action returns
        // (e.g., manipulating the resultContext.Result)
    }
}
```

Applying the Filter

Because the filter requires dependency injection (the repository), it cannot be applied simply as `[ValidateVehicleExistsFilter]`. It must be applied using `[ServiceFilter]`, and the filter itself must be registered in the DI container.

```
// 1. In Program.cs
builder.Services.AddScoped<ValidateVehicleExistsFilter>();

// 2. In the Controller
[ApiController]
[Route("api/[controller]")]
public class VehiclesController : ControllerBase
{
    [HttpGet("{id}")]
    [ServiceFilter(typeof(ValidateVehicleExistsFilter))] // Applies the filter
    public IActionResult GetVehicle(int id)
    {
        // This code is only reached if the filter verifies the vehicle exists.
        return Ok(...);
    }
}
```